

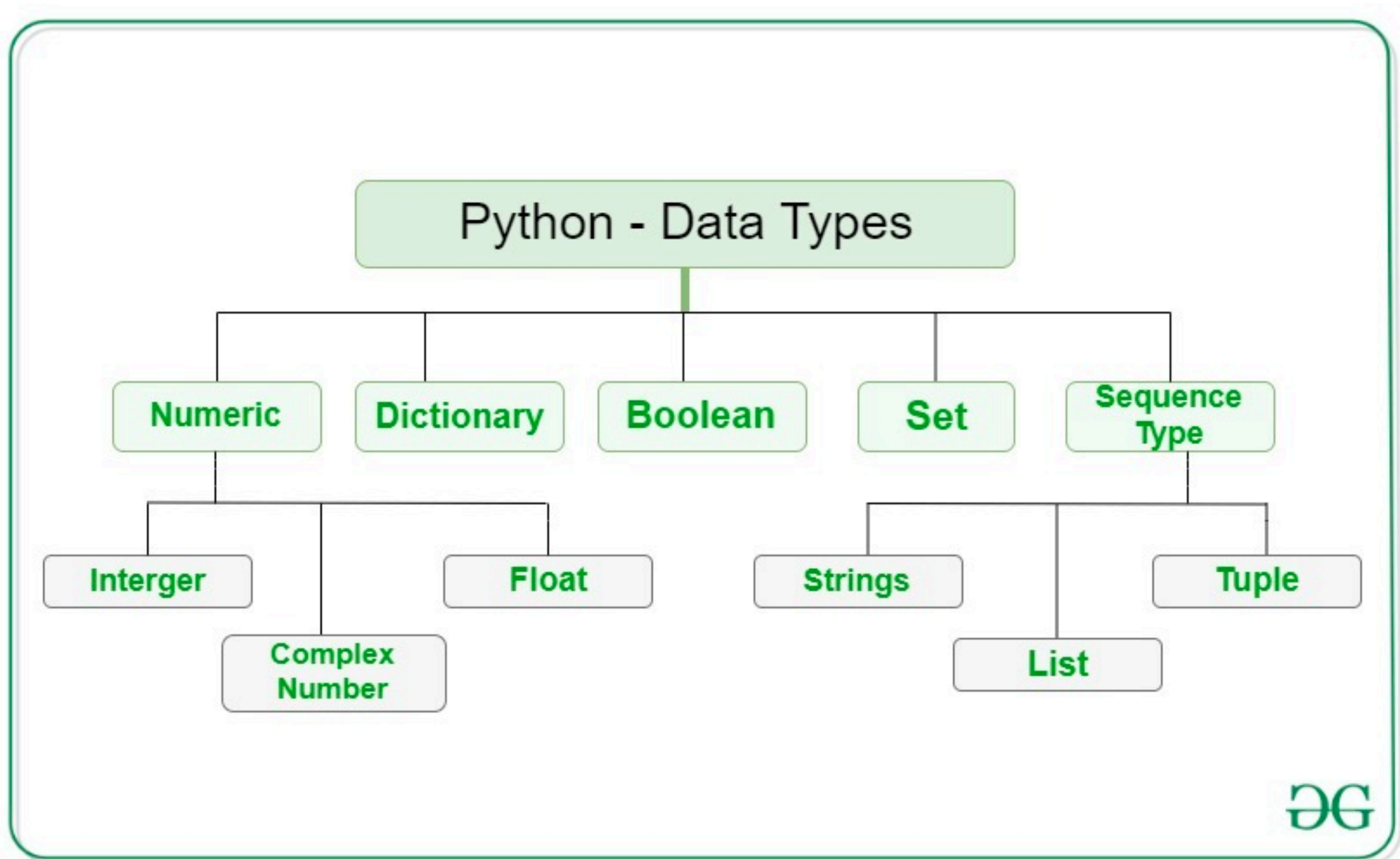
Fundamentos de programación II

Alex Di Genova

29/09/2025

¿Qué frases entienden los computadores?

Explicación del concepto variable.



Manipulación de archivos

```
file=open("file.txt")  
for line in file:  
    print(line)  
file.close()
```

```
file=open("newfile.txt","w")  
file.write("hola archivo\n")  
file.close()
```

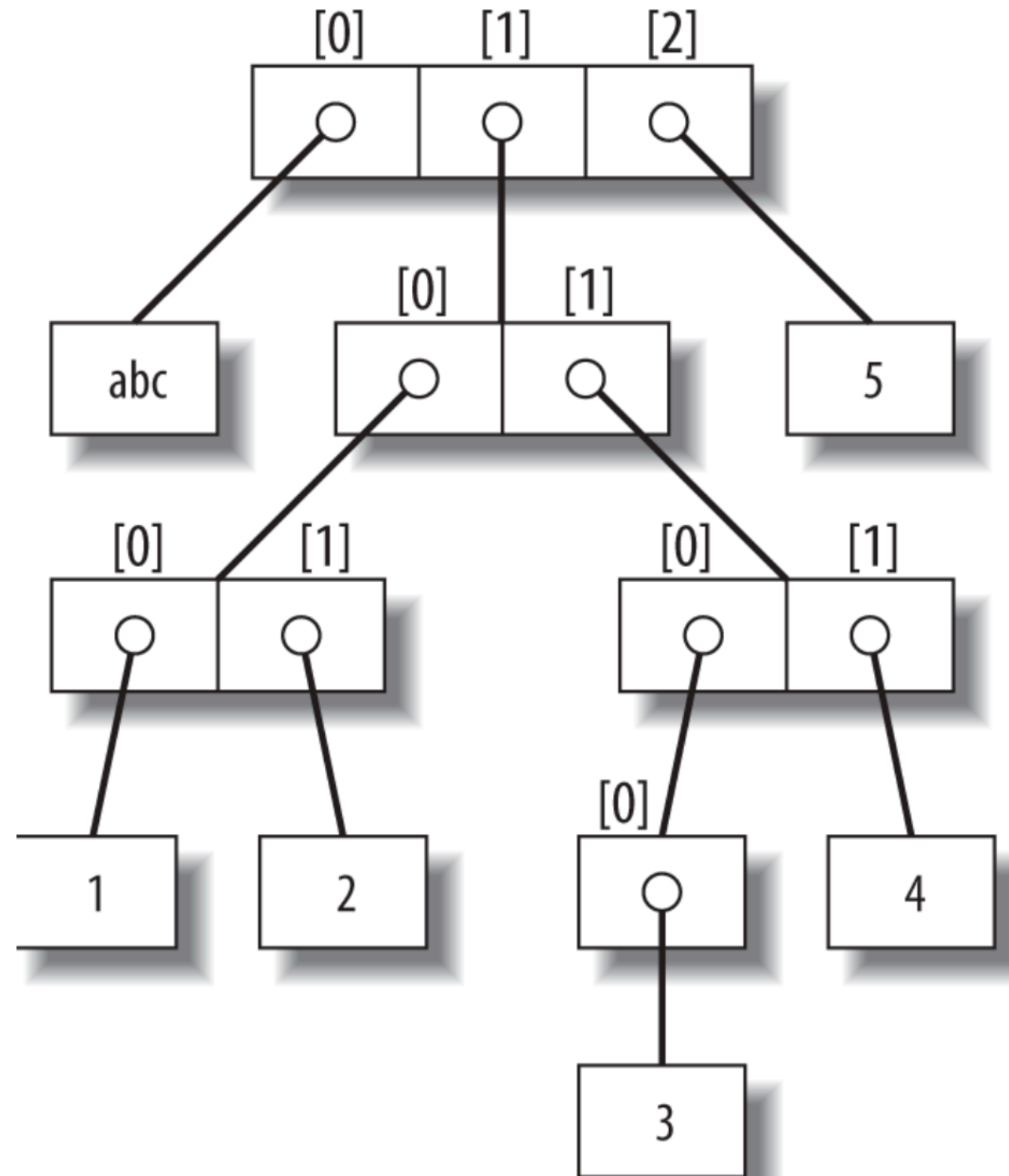
```
for line in open("file.txt"):  
    line.rstrip()  
    print(line)
```

```
print(open("file.txt").read())
```

Estructuras complejas

L = ['abc', [(1, 2), ([3], 4)], 5]

- Como obtengo el valor abc?
- Como obtengo el valor 2?
- Como obtengo el valor 5?
- Como obtengo el valor 1?
- Como obtengo el valor 3?
- En python listas, diccionarios y tuplas pueden guardar cualquier objeto.

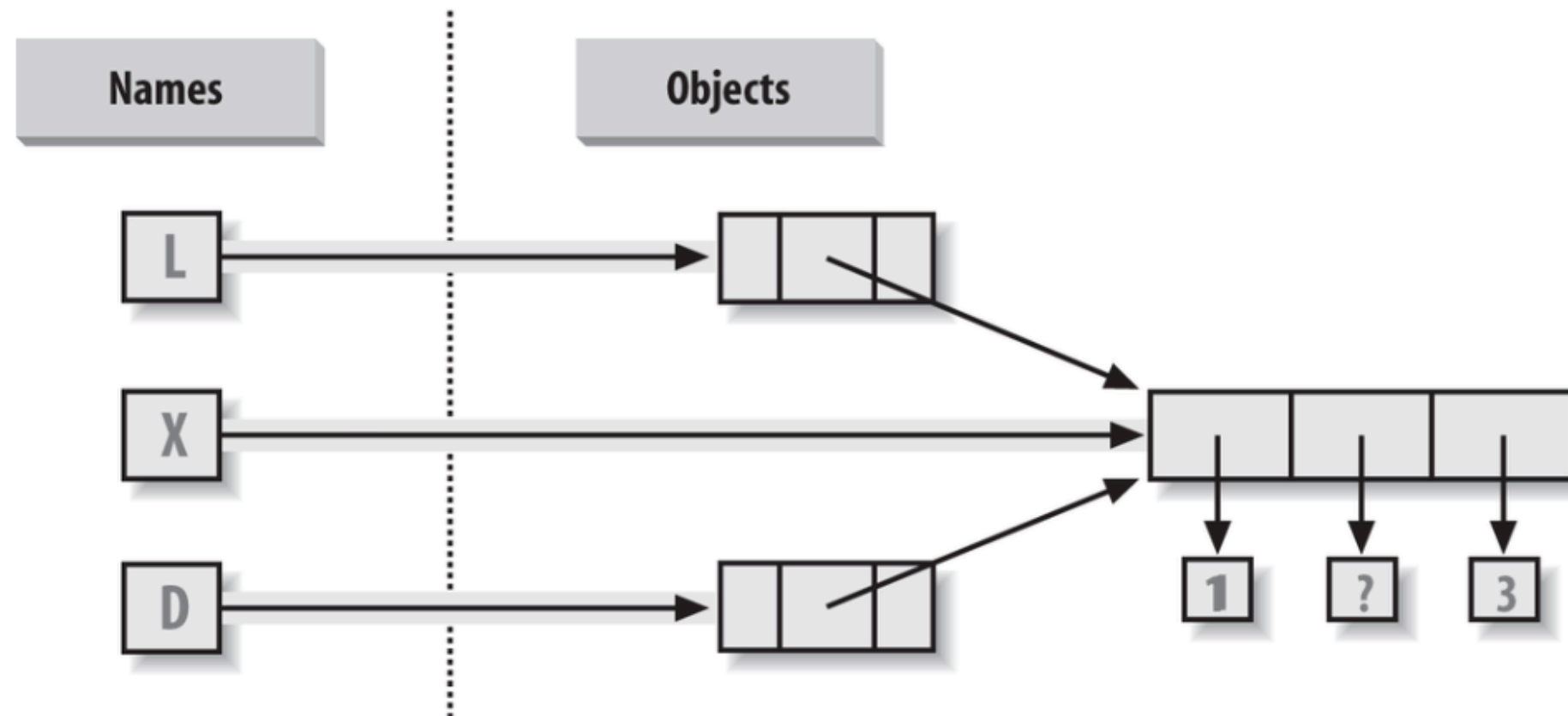


Referencia vs copias

`X = [1,2,3]`

`L=["a",X,"b"]`

`D={'x':X, 'y':2}`



`X[1] = "sorpresa"`

`print(L) ?`

`print(D) ?`

Copiar objetos

Lista

`L.copy()`

Diccionario

`D.copy()`

Objetos complejos

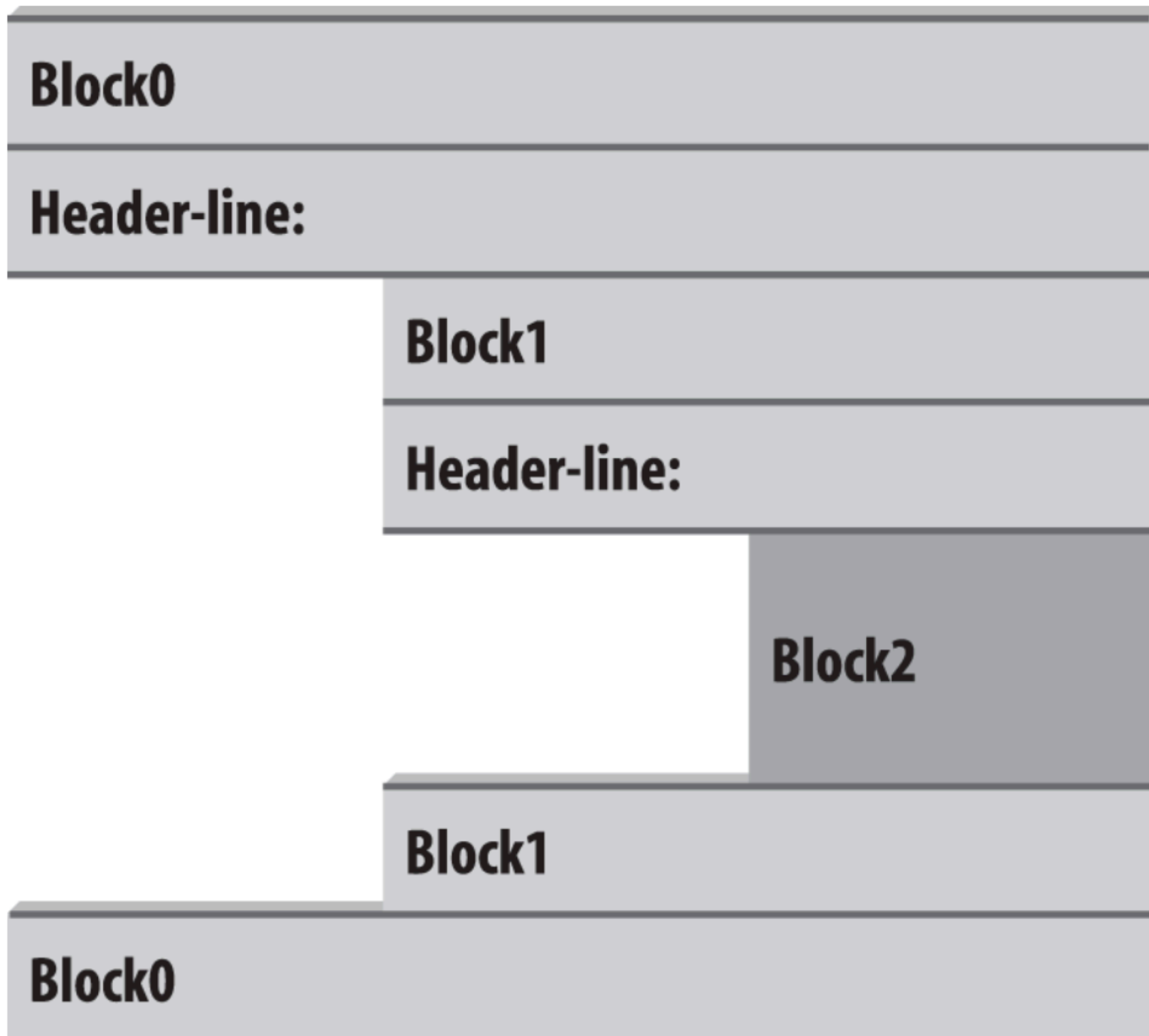
`#L = ['abc', [(1, 2), ([3], 4)], 5]`

Import copy

`X=copy.deepcopy(L)`

Reglas de indentación

Bloques de código



Programación Orientada a Objetos

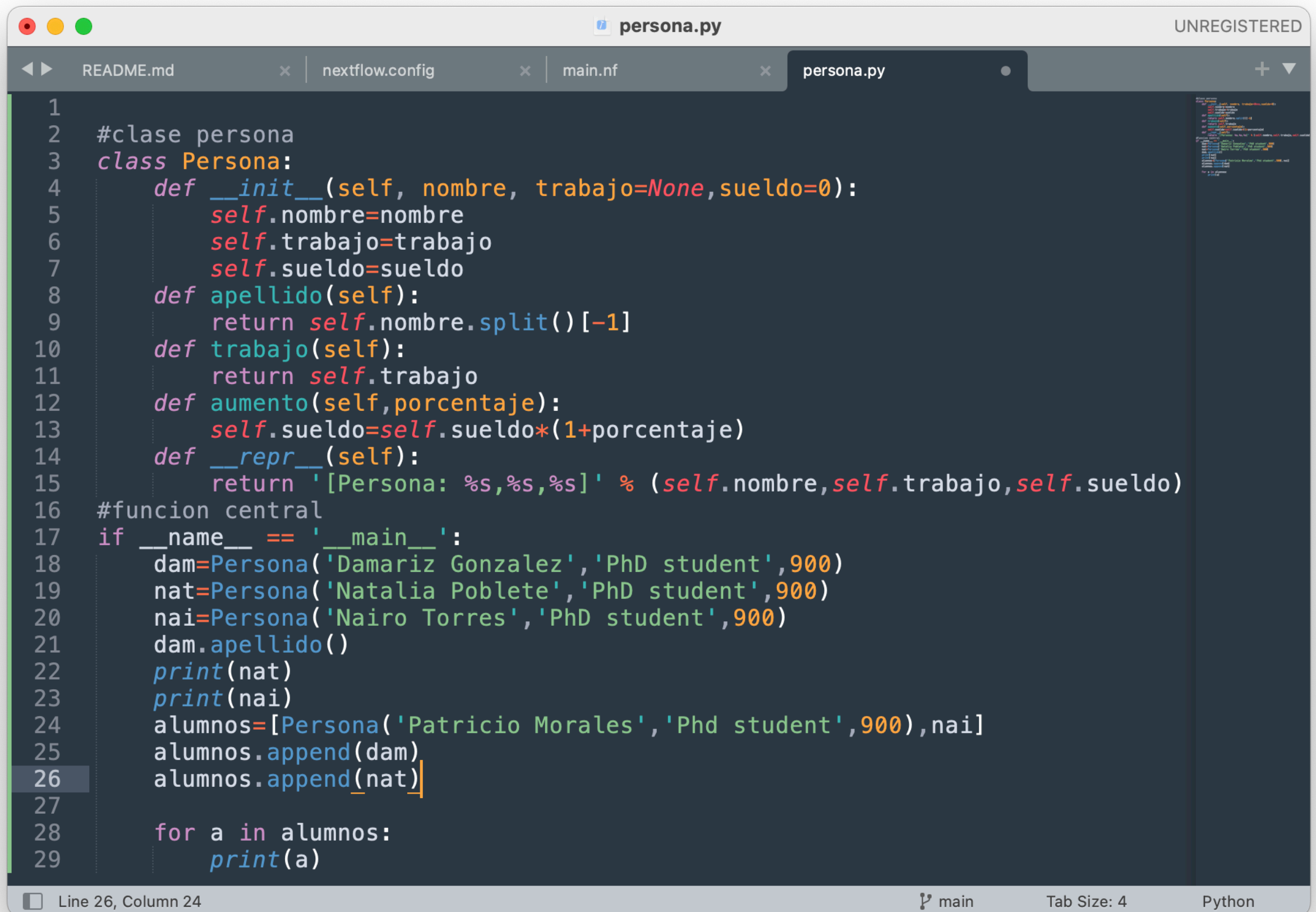
OOP

```
1
2 #clase persona
3 class Persona:
4     def __init__(self, nombre, trabajo=None, sueldo=0):
5         self.nombre=nombre
6         self.trabajo=trabajo
7         self.sueldo=sueldo
8     def apellido(self):
9         return self.nombre.split()[-1]
10    def trabajo(self):
11        return self.trabajo
12    def aumento(self, porcentaje):
13        self.sueldo=self.sueldo*(1+porcentaje)
14    def __repr__(self):
15        return '[Persona: %s,%s,%s]' % (self.nombre,self.trabajo,self.sueldo)
16
17 if __name__ == '__main__': ...|
```

- Mi primera clase

Programación Orientada a Objetos

OOP



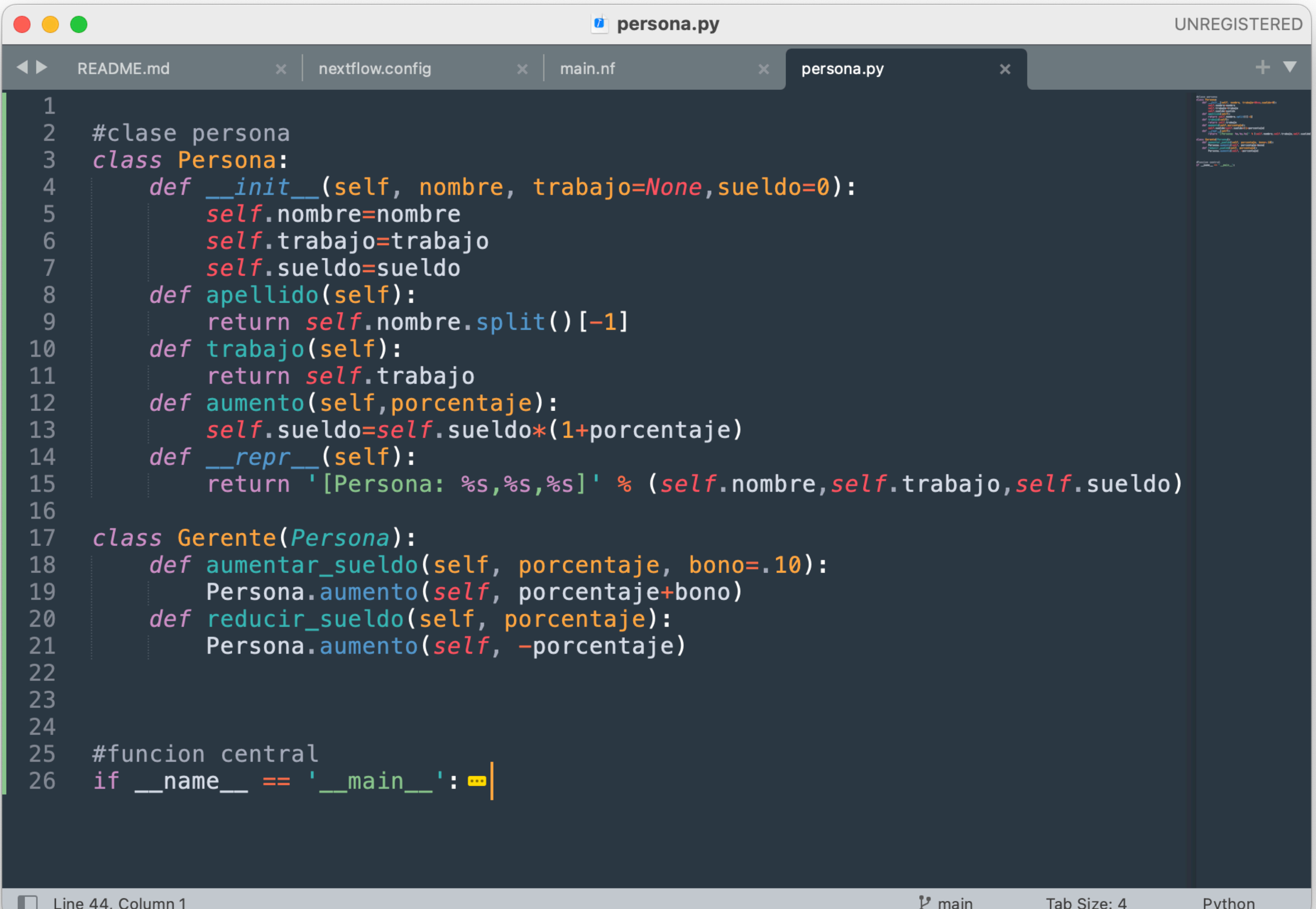
```
1
2 #clase persona
3 class Persona:
4     def __init__(self, nombre, trabajo=None, sueldo=0):
5         self.nombre=nombre
6         self.trabajo=trabajo
7         self.sueldo=sueldo
8     def apellido(self):
9         return self.nombre.split()[-1]
10    def trabajo(self):
11        return self.trabajo
12    def aumento(self, porcentaje):
13        self.sueldo=self.sueldo*(1+porcentaje)
14    def __repr__(self):
15        return '[Persona: %s,%s,%s]' % (self.nombre,self.trabajo,self.sueldo)
16 #funcion central
17 if __name__ == '__main__':
18     dam=Persona('Damariz Gonzalez','PhD student',900)
19     nat=Persona('Natalia Poblete','PhD student',900)
20     nai=Persona('Nairo Torres','PhD student',900)
21     dam.apellido()
22     print(nat)
23     print(nai)
24     alumnos=[Persona('Patricio Morales','Phd student',900),nai]
25     alumnos.append(dam)
26     alumnos.append(nat)
27
28     for a in alumnos:
29         print(a)
```

Line 26, Column 24

main Tab Size: 4 Python

Programación Orientada a Objetos

Herencia



```
1
2 #clase persona
3 class Persona:
4     def __init__(self, nombre, trabajo=None, sueldo=0):
5         self.nombre=nombre
6         self.trabajo=trabajo
7         self.sueldo=sueldo
8     def apellido(self):
9         return self.nombre.split()[-1]
10    def trabajo(self):
11        return self.trabajo
12    def aumento(self, porcentaje):
13        self.sueldo=self.sueldo*(1+porcentaje)
14    def __repr__(self):
15        return '[Persona: %s,%s,%s]' % (self.nombre, self.trabajo, self.sueldo)
16
17    class Gerente(Persona):
18        def aumentar_sueldo(self, porcentaje, bono=.10):
19            Persona.aumento(self, porcentaje+bono)
20        def reducir_sueldo(self, porcentaje):
21            Persona.aumento(self, -porcentaje)
22
23
24
25 #funcion central
26 if __name__ == '__main__':
```

Line 44, Column 1

main Tab Size: 4 Python

README.md

nextflow.config

main.nf

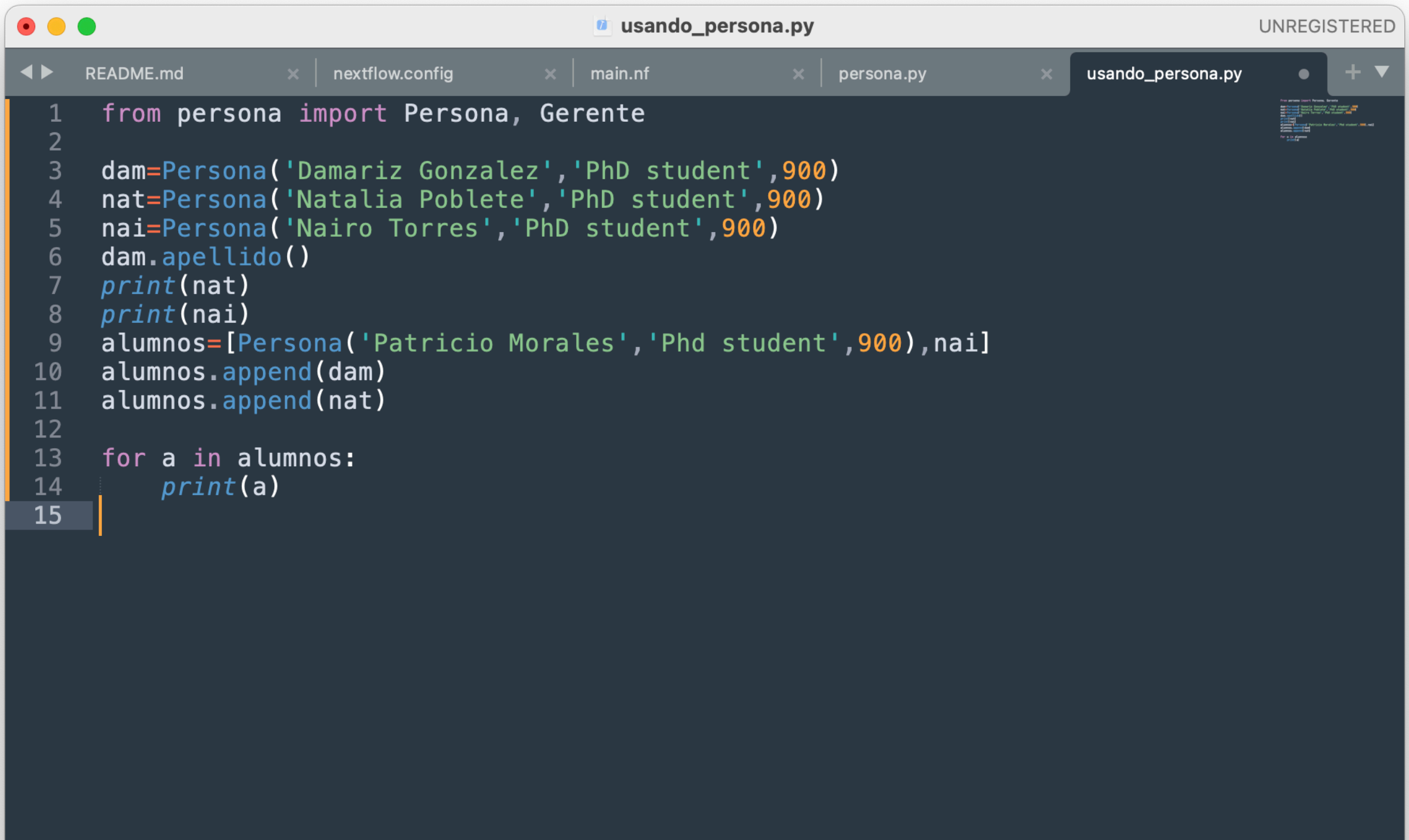
persona.py

```
1
2 #clase persona
3 class Persona:
16
17 class Gerente(Persona):
18     def aumentar_sueldo(self, porcentaje, bono=.10):
19         Persona.aumento(self, porcentaje+bono)
20     def reducir_sueldo(self, porcentaje):
21         Persona.aumento(self, -porcentaje)
22
23
24
25 #funcion central
26 if __name__ == '__main__':
27     dam=Persona('Damariz Gonzalez','PhD student',900)
28     nat=Persona('Natalia Poblete','PhD student',900)
29     nai=Persona('Nairo Torres','PhD student',900)
30     dam.apellido()
31     print(nat)
32     print(nai)
33     alumnos=[Persona('Patricio Morales','Phd student',900),nai]
34     alumnos.append(dam)
35     alumnos.append(nat)
36
37     for a in alumnos:
38         print(a)
39
40     mauro=Gerente('Mauricio Avendaño','Gerente personas',2000)
41     mauro.reducir_sueldo(.25)
42
43     print(mauro)
```

[Ajustar/reutilizar Código](#)

Programación Orientada a Objetos

Utilizando la clase desde otro programa



```
1  from persona import Persona, Gerente
2
3  dam=Persona('Damariz Gonzalez','PhD student',900)
4  nat=Persona('Natalia Poblete','PhD student',900)
5  nai=Persona('Nairo Torres','PhD student',900)
6  dam.apellido()
7  print(nat)
8  print(nai)
9  alumnos=[Persona('Patricio Morales','Phd student',900),nai]
10 alumnos.append(dam)
11 alumnos.append(nat)
12
13 for a in alumnos:
14     print(a)
15
```